Generative Adversarial Privacy

Honghui Xu, Zuobin Xiong and Zhipeng Cai
Department of Computer Science
Georgia State University

Copyright © 2021 Honghui Xu, Georgia State University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755.

Contents

1 Introduction **1**

2 Lab Preparation **2**

3 Privacy Attack with GANs **4**

4 Privacy Protection with GANs **15**

5 General Questions **29**

1 Introduction

This document servers as a experiment guidance for the purpose of using Generative Adversarial Networks (GAN) on privacy attack and privacy protection.

1.1 Generative Adversarial Networks

As the most creative idea of deep learning in recent years, Generative Adversarial Networks (GANs) have been widely applied in the field of adversarial learning since it was proposed in 2014. GANs consist of two "adversarial" models, including a generator G and a discriminator D. These two adversarial models play with each other to complete in a min-max game, where G intentionally generates samples from a real data distribution to fool D while D judges whether its input is the fake data generated by G or the real data. The generator G is built as a deep neural network that can be designed in various ways, and the discriminator D is also a neural network learned by the traditional supervised learning methods that classifies inputs into two sets (*i.e.*, fake or real).

Mathematically speaking, G can be expressed in any form but a simple differentiable function, and $G(z)$ is an output sample drawn from p_g, where z is a low-dimensional vector sampled from a prior distribution p_z. The goal of D is to classify the data from $G(z)$ as fake and the data from training set p_{data} as real. Accordingly, GANs can be formulated as a structured probabilistic model to optimize the following loss function:

$$\begin{aligned}\min_G \max_D \mathcal{L}_{GAN}(G, D) =& \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)]+ \\ & \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))],\end{aligned} \tag{1}$$

where G aims to minimize $\mathcal{L}_{GAN}(G, D)$, while D aims to maximize it. Actually, in practical training process, the expectation substitute to mean average of some mini batches because there only exist limited data. And in each mini batch, the loss function is minimized and maximized alternatively by using gradient descent method.

Furthermore, GANs can be extended to a conditional version with an additional input y that could be any kind of auxiliary information (*e.g.*, class labels or data from other domains). The objective function of conditional GANs is expressed as:

$$\begin{aligned}\min_G \max_D \mathcal{L}_{GAN}(G, D) =& \mathbb{E}_{x\sim p_{data}(x)}[\log D(x|y)]+ \\ & \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z|y)))].\end{aligned} \tag{2}$$

In our hands on experiments, GAN-based idea is exploited to configure attack and defense modules. Details about experiments can be found in Section 3 and Section 4.

2 Lab Preparation

We will do the lab project on Google's Colaboratory https://colab.research.google.com. Everyone can log into the Colab with your own google account. In this section, we will simply introduce the basic functions of Google Golab platform, which will be used in our experiments. We only show the necessary steps and setup commands related to open and create a notebook on Colab, more details about concrete implementation of codes will be displayed in following sections. If you are very interested in deep learning applications, you can learn more from Youtube link https://www.youtube.com/watch?v=inN8seMm7UI about how to use Colab to train more machine learning models.

2.1 New a Notebook to Start

First of all, we need to new a notebook for code programming in Colab, shown in Fig. 1. You can name it as "Generative Adversarial Privacy" (GAP for short) or any other name you want to specify this notebook file.

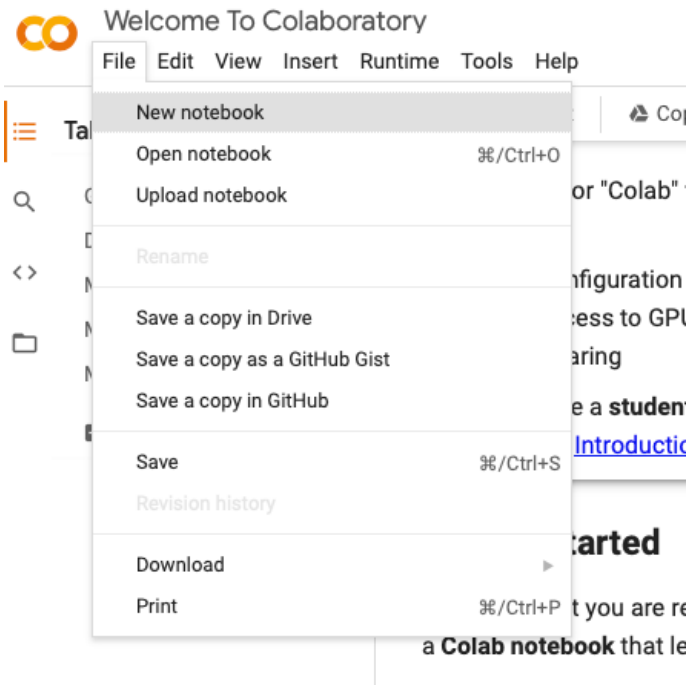
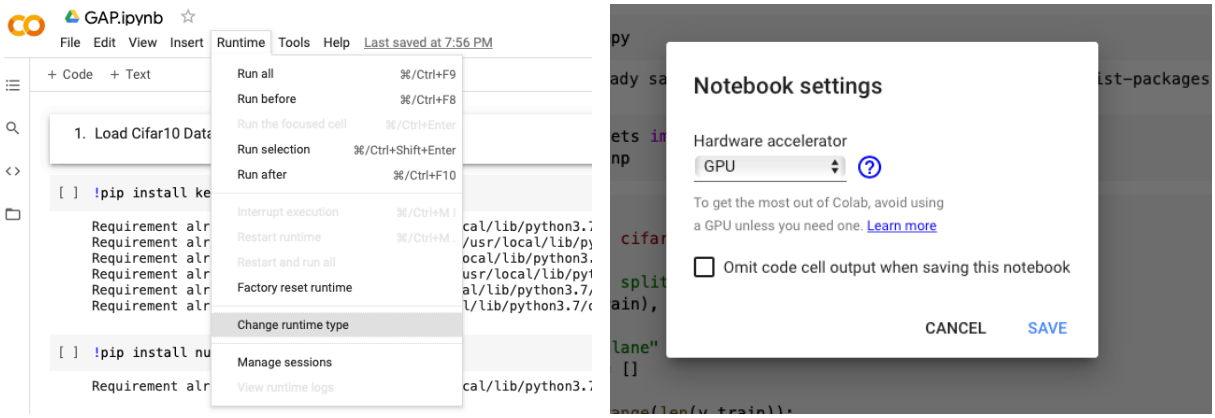


Figure 1: New a Notebook in Colab.

2.2 Set a Runtime Type

After we enter the new notebook, we should set a runtime type for code running. Deep learning models usually is implemented on GPUs. In Colab, we can use Google's online GPUs resource. You firstly need to find "change runtime type", presented in Fig. 2a, and then choose "GPU" type for running codes, shown in Fig. 2b.



(a) Change Runtime Type. (b) Choose GPU Type.

Figure 2: Select Runtime Type.

2.3 Check Environment

After choosing the correct GPU or TPU type for experiment, it's time to check if the selected GPU/TPU is suitable with our desire library version or deep learning frameworks. There are a lot of available open source deep learning library and frameworks in the Internet, which may requires different versions of library. Using incompatible library will cause unpredictable error or even system crash. For example, if the project we are going to implement requires Python 3.6 or higher, and use Pytorch 1.4 or higher, we can check the versions via the following commands:

```
import sys
print(sys.version,"\n")

import torch
print(torch.__version__,"\n")

import tensorflow
print(tensorflow.__version__)
```

```
3.7.10 (default, May  3 2021, 02:48:31)
[GCC 7.5.0]

1.8.1+cu101

2.4.1
```

Figure 3: Check your library version.

3 Privacy Attack with GANs

The idea of privacy attack with GANs comes from a reference paper "Generating adversarial examples with adversarial networks". Reference: https://dl.acm.org/doi/abs/10.5555/3304222.3304312

3.1 Data Preparation

We use deep learning framework Pytorch for the implementation of privacy attack with GAN. The data we use to illustrate this experiment can be downloaded from Pytorch libraty by torchvision.dataset. Necessary code is below:

```
import torch
import torchvision.datasets
import torchvision.transforms as transforms
mnist_dataset = torchvision.datasets.MNIST('./dataset', train=True,

transform=transforms.ToTensor(), download=True)
```

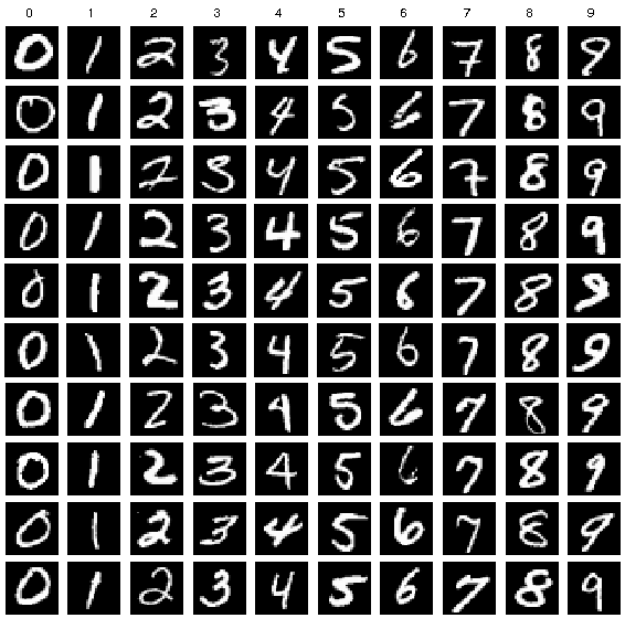


Figure 4: Visualization of downloaded data.

3.2 Training The Target Model

After loading the training data, we need to use the data to train a classifier model, which is what we call the target model. The target model is an oracle that can predict the label of any given input data. It is also the model we aim at invading. Simply speaking, if the target model is trained well, our purpose of privacy attack is to use GAN-based idea to generate some adversarial example to make the target model mis-classify them. The core part of training target model is below:

```
# Target Model definition
class MNIST_target_net(nn.Module):
def __init__(self):
super(MNIST_target_net, self).__init__()
self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
self.conv2 = nn.Conv2d(32, 32, kernel_size=3)
self.conv3 = nn.Conv2d(32, 64, kernel_size=3)
self.conv4 = nn.Conv2d(64, 64, kernel_size=3)

self.fc1 = nn.Linear(64*4*4, 200)
self.fc2 = nn.Linear(200, 200)
self.logits = nn.Linear(200, 10)

def forward(self, x):
x = F.relu(self.conv1(x))
x = F.relu(self.conv2(x))
```

```
x = F.max_pool2d(x, 2)
x = F.relu(self.conv3(x))
x = F.relu(self.conv4(x))
x = F.max_pool2d(x, 2)
x = x.view(-1, 64*4*4)
x = F.relu(self.fc1(x))
x = F.dropout(x, 0.5)
x = F.relu(self.fc2(x))
x = self.logits(x)
return x


# training the target model
target_model = MNIST_target_net().to(device)
target_model.train()
opt_model = torch.optim.Adam(target_model.parameters(), lr=0.001)
epochs = 40
for epoch in range(epochs):
loss_epoch = 0
if epoch == 20:
opt_model = torch.optim.Adam(target_model.parameters(), lr=0.0001)
for i, data in enumerate(train_dataloader, 0):
train_imgs, train_labels = data
train_imgs, train_labels = train_imgs.to(device), train_labels.to(device)
logits_model = target_model(train_imgs)
loss_model = F.cross_entropy(logits_model, train_labels)
loss_epoch += loss_model
opt_model.zero_grad()
loss_model.backward()
opt_model.step()

print('loss in epoch %d: %f' % (epoch, loss_epoch.item()))

# save model
targeted_model_file_name = './MNIST_target_model.pth'
torch.save(target_model.state_dict(), targeted_model_file_name)
target_model.eval()

# MNIST test dataset
```

```
mnist_dataset_test = torchvision.datasets.MNIST('./dataset',

train=False, transform=transforms.ToTensor(), download=True)
test_dataloader = DataLoader(mnist_dataset_test,

batch_size=batch_size, shuffle=True, num_workers=1)
num_correct = 0
for i, data in enumerate(test_dataloader, 0):
test_img, test_label = data
test_img, test_label = test_img.to(device), test_label.to(device)
pred_lab = torch.argmax(target_model(test_img), 1)
num_correct += torch.sum(pred_lab==test_label,0)

print('accuracy in testing set: %f
'%(num_correct.item()/len(mnist_dataset_test)))
```

3.3 Train The Attack Model

In this section, we will use generative adversarial networks to achieve the goal of privacy attack for images. This method is usually name adversarial example generation by GANs. Specifically, the generator G in GANs is built to generate a imperceptible noise to add on original images, which will cause the target model get wrong predictions. While the discriminator is built to regulate the generated noise not exceeding certain threshold, also the original image and the generated image should be similar. In the following, we will go through the module building and training of generative adversarial example.

3.3.1 Build a Generator

We use Pytorch deep learning framework to build our generator. The generator usually is designed as an neural network that takes a random noise as input then output a data that has the same dimension as the target image data. Here, use the convolutional layer and resnetBlock layer to build the generator. The convolutional and resnetblock layer are very popular structure in neural network. For more details about the network structure, please use this link to explore. https://en.wikipedia.org/wiki/Convolutional_neural_network.

The following code defines the generator:

```
class Generator(nn.Module):
def __init__(self,
gen_input_nc,
image_nc,
):
```

```
super(Generator, self).__init__()

encoder_lis = [
# MNIST:1*28*28
nn.Conv2d(gen_input_nc, 8, kernel_size=3, stride=1, padding=0, bias=True),
nn.InstanceNorm2d(8),
nn.ReLU(),
# 8*26*26
nn.Conv2d(8, 16, kernel_size=3, stride=2, padding=0, bias=True),
nn.InstanceNorm2d(16),
nn.ReLU(),
# 16*12*12
nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=0, bias=True),
nn.InstanceNorm2d(32),
nn.ReLU(),
# 32*5*5
]

bottle_neck_lis = [ResnetBlock(32),
ResnetBlock(32),
ResnetBlock(32),
ResnetBlock(32),]

decoder_lis = [
nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=0, bias=False),
nn.InstanceNorm2d(16),
nn.ReLU(),
# state size. 16 x 11 x 11
nn.ConvTranspose2d(16, 8, kernel_size=3, stride=2, padding=0, bias=False),
nn.InstanceNorm2d(8),
nn.ReLU(),
# state size. 8 x 23 x 23
nn.ConvTranspose2d(8, image_nc, kernel_size=6, stride=1, padding=0, bias=False),
nn.Tanh()
# state size. image_nc x 28 x 28
]

self.encoder = nn.Sequential(*encoder_lis)
self.bottle_neck = nn.Sequential(*bottle_neck_lis)
```

```
self.decoder = nn.Sequential(*decoder_lis)

def forward(self, x):
x = self.encoder(x)
x = self.bottle_neck(x)
x = self.decoder(x)
return x
```

3.3.2 Build a Discriminator

The discriminator D is also constructed by convolutional layer, which takes an image data as input then output a scalar value in the range [0,1]. As the original generative adversarial network, we design D as a binary classifier, where the real image data is classified as label 1 and the generated adversarial example is classified as label 0. The code below describes the implementation of discriminator:

```
class Discriminator(nn.Module):
def __init__(self, image_nc):
super(Discriminator, self).__init__()
# MNIST: 1*28*28
model = [
nn.Conv2d(image_nc, 8, kernel_size=4, stride=2, padding=0, bias=True),
nn.LeakyReLU(0.2),
# 8*13*13
nn.Conv2d(8, 16, kernel_size=4, stride=2, padding=0, bias=True),
nn.BatchNorm2d(16),
nn.LeakyReLU(0.2),
# 16*5*5
nn.Conv2d(16, 32, kernel_size=4, stride=2, padding=0, bias=True),
nn.BatchNorm2d(32),
nn.LeakyReLU(0.2),
nn.Conv2d(32, 1, 1),
nn.Sigmoid()
# 32*1*1
]
self.model = nn.Sequential(*model)

def forward(self, x):
output = self.model(x).squeeze()
return output
```

The above structure has only 4 convolution layer, which provides a easy binary classifier. We didn't specify the hyperparameter tuning process in this project. If you want to have a perfect-performed GAN structure, you should try different parameters combination by yourself to find the best fit.

3.3.3 Train the Attack Model

After defining the generator and discriminator, no we can start to train the generative adversarial example. There are several steps in this training process, we will illustrate them one by one:

(i) At first, we define some training related parameter, including what device we are using, where is the target model location, and directory for model and data.

```
use_cuda=True
image_nc=1
epochs = 60
batch_size = 128
BOX_MIN = 0
BOX_MAX = 1
# Define what device we are using
print("CUDA Available: ",torch.cuda.is_available())
device = torch.device("cuda" if (use_cuda and torch.cuda.is_available()) else "cpu")

pretrained_model = "./MNIST_target_model.pth"
targeted_model = MNIST_target_net().to(device)
targeted_model.load_state_dict(torch.load(pretrained_model))
targeted_model.eval()
model_num_labels = 10
```

(ii) Then, after finishing these global parameters, we define a training batch operation, which describes what should be done in one batch data:

```
def train_batch(self, x, labels):
# optimize D
for i in range(1):
perturbation = self.netG(x)

# add a clipping trick
adv_images = torch.clamp(perturbation, -0.3, 0.3) + x
adv_images = torch.clamp(adv_images, self.box_min, self.box_max)

self.optimizer_D.zero_grad()
```

```
pred_real = self.netDisc(x)
loss_D_real = F.mse_loss(pred_real, torch.ones_like(pred_real, device=self.device))
loss_D_real.backward()

pred_fake = self.netDisc(adv_images.detach())
loss_D_fake = F.mse_loss(pred_fake, torch.zeros_like(pred_fake, device=self.device))
loss_D_fake.backward()
loss_D_GAN = loss_D_fake + loss_D_real
self.optimizer_D.step()

# optimize G
for i in range(1):
self.optimizer_G.zero_grad()

# cal G's loss in GAN
pred_fake = self.netDisc(adv_images)
loss_G_fake = F.mse_loss(pred_fake, torch.ones_like(pred_fake, device=self.device))
loss_G_fake.backward(retain_graph=True)

# calculate perturbation norm
C = 0.1
loss_perturb = torch.mean(torch.norm(perturbation.view(perturbation.shape[0], -1), 2, dim=1))
# loss_perturb = torch.max(loss_perturb - C, torch.zeros(1, device=self.device))

# cal adv loss
logits_model = self.model(adv_images)
probs_model = F.softmax(logits_model, dim=1)
onehot_labels = torch.eye(self.model_num_labels, device=self.device)[labels]

# C&W loss function
real = torch.sum(onehot_labels * probs_model, dim=1)
other, _ = torch.max((1 - onehot_labels) * probs_model - onehot_labels * 10000, dim=1)
zeros = torch.zeros_like(other)
loss_adv = torch.max(real - other, zeros)
loss_adv = torch.sum(loss_adv)

# maximize cross_entropy loss
# loss_adv = -F.mse_loss(logits_model, onehot_labels)
# loss_adv = - F.cross_entropy(logits_model, labels)
```

```
adv_lambda = 10
pert_lambda = 1
loss_G = adv_lambda * loss_adv + pert_lambda * loss_perturb
loss_G.backward()
self.optimizer_G.step()

return loss_D_GAN.item(), loss_G_fake.item(), loss_perturb.item(), loss_adv.item()
```

In the training batch, we input an original image into generator G and produces a noise perturbation pattern; then we clip the noise scale into a predefined norm and add it on original images as generated adversarial example; next step, the real image and generated adversarial example are input to discriminator D to get the corresponding loss value; finally, both G and D are updated through gradient descent method based on the loss function value.

(iii) the generator and discriminator are initialized from scratch. At the beginning of training, there is an extra step called weight initialization, which is done by the following function:

```
def weights_init(m):
classname = m.__class__.__name__
if classname.find('Conv') != -1:
nn.init.normal_(m.weight.data, 0.0, 0.02)
elif classname.find('BatchNorm') != -1:
nn.init.normal_(m.weight.data, 1.0, 0.02)
nn.init.constant_(m.bias.data, 0)
```

(iv) finally, after all above processes are prepared well, we can start to train the GAN-based generative adversarial example method on the entire data. Here we use a dynamically changed learning rate, when epoch is less than 50, the learning rate is set to be 0.0001; when epoch is greater than 80, we set the learning rate to be 0.00001. This dynamic learning rate helps model converge faster and achieve better result.

```
def train(self, train_dataloader, epochs):
for epoch in range(1, epochs+1):

if epoch == 50:
self.optimizer_G = torch.optim.Adam(self.netG.parameters(),
lr=0.0001)
self.optimizer_D = torch.optim.Adam(self.netDisc.parameters(),
lr=0.0001)
if epoch == 80:
self.optimizer_G = torch.optim.Adam(self.netG.parameters(),
```

```
lr=0.00001)
self.optimizer_D = torch.optim.Adam(self.netDisc.parameters(),
lr=0.00001)
loss_D_sum = 0
loss_G_fake_sum = 0
loss_perturb_sum = 0
loss_adv_sum = 0
for i, data in enumerate(train_dataloader, start=0):
images, labels = data
images, labels = images.to(self.device), labels.to(self.device)

loss_D_batch, loss_G_fake_batch, loss_perturb_batch, loss_adv_batch =  self.train_batch(images
loss_D_sum += loss_D_batch
loss_G_fake_sum += loss_G_fake_batch
loss_perturb_sum += loss_perturb_batch
loss_adv_sum += loss_adv_batch

# print statistics
num_batch = len(train_dataloader)
print("epoch d:
loss_D: %.3f, loss_G_fake: %.3f,
loss_perturb: %.3f, loss_adv: %.3f,
" %
(epoch, loss_D_sum/num_batch, loss_G_fake_sum/num_batch,
loss_perturb_sum/num_batch, loss_adv_sum/num_batch))

# save generator
if epoch%20==0:
netG_file_name = models_path + 'netG_epoch_' + str(epoch) + '.pth'
torch.save(self.netG.state_dict(), netG_file_name)
```

3.4 Attack Performance Evaluation

The above training process will produce a generator G^*, which is exactly what we are pursuing. Now we can use the generator G^* to produce numerous adversarial example quickly by using $x' = x + G^*(x)$, where x is the target real data we want to invade, and x' is the generated adversarial example.

We can pick the stored model G from the path "MNIST_target_model.pth" and then evaluate its performance on training dataset and testing dataset separately as shown in the following code:

```
# load the pretrained model
```

```
pretrained_model = "./MNIST_target_model.pth"
target_model = MNIST_target_net().to(device)
target_model.load_state_dict(torch.load(pretrained_model))
target_model.eval()

# load the generator of adversarial examples
pretrained_generator_path = './models/netG_epoch_60.pth'
pretrained_G = models.Generator(gen_input_nc, image_nc).to(device)
pretrained_G.load_state_dict(torch.load(pretrained_generator_path))
pretrained_G.eval()

# test adversarial examples in MNIST training dataset
mnist_dataset = torchvision.datasets.MNIST('./dataset',

train=True, transform=transforms.ToTensor(), download=True)
train_dataloader = DataLoader(mnist_dataset, batch_size

=batch_size, shuffle=False, num_workers=1)
num_correct = 0
for i, data in enumerate(train_dataloader, 0):
test_img, test_label = data
test_img, test_label = test_img.to(device), test_label.to(device)
perturbation = pretrained_G(test_img)
perturbation = torch.clamp(perturbation, -0.3, 0.3)
adv_img = perturbation + test_img
adv_img = torch.clamp(adv_img, 0, 1)
pred_lab = torch.argmax(target_model(adv_img),1)
num_correct += torch.sum(pred_lab==test_label,0)

print('MNIST training dataset:')
print('num_correct: ', num_correct.item())
print('accuracy of adv imgs in training set: %f
'%(num_correct.item()/len(mnist_dataset)))

# test adversarial examples in MNIST testing dataset
mnist_dataset_test = torchvision.datasets.MNIST('./dataset',
train=False, transform=transforms.ToTensor(), download=True)
test_dataloader = DataLoader(mnist_dataset_test, batch_size
=batch_size, shuffle=False, num_workers=1)
```

```
num_correct = 0
for i, data in enumerate(test_dataloader, 0):
test_img, test_label = data
test_img, test_label = test_img.to(device), test_label.to(device)
perturbation = pretrained_G(test_img)
perturbation = torch.clamp(perturbation, -0.3, 0.3)
adv_img = perturbation + test_img
adv_img = torch.clamp(adv_img, 0, 1)
pred_lab = torch.argmax(target_model(adv_img),1)
num_correct += torch.sum(pred_lab==test_label,0)

print('num_correct: ', num_correct.item())
print('accuracy of adv imgs in testing set: %f
'%(num_correct.item()/len(mnist_dataset_test)))
```

Finally, the evaluation process will output what is the attack success rate of our trained generative adversarial examples.

4 Privacy Protection with GANs

The idea of privacy protection with GANs comes from a reference paper "Protecting Visual Secrets Using Adversarial Nets". Reference: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8014908

4.1 Data Preparation

We will use keras (a open-sourced deep learning kit https://keras.io) for our code programming. Thus, we should install keras by using the following command.

```
!pip install keras
```

Since we should do some numrical operation on array, we also need to install numpy package in python by using the following command.

```
!pip install numpy
```

(Note: i) we just need to click +code in the webpage to add one new code block. ii) we just need to click +text in the webpage to add one new note. iii) we just click the "run" button to run the code in one block. These three buttons are shown in Fig. 5.)

4.2 Load Public Data

We load cifar10 dataset (a benchmark image dataset) in keras as public data without privacy. Before load dataset, we should first import two packages as below:

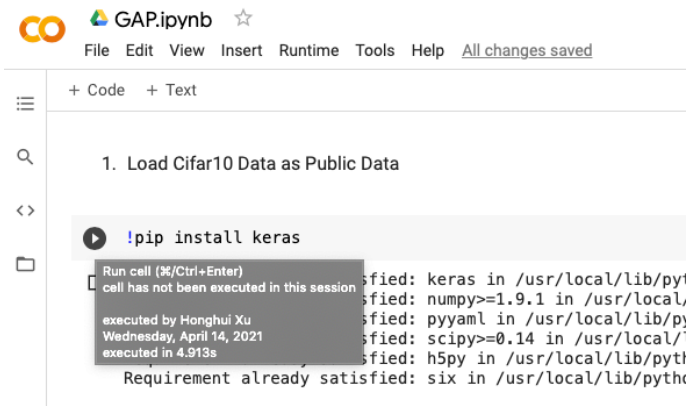


Figure 5: Three Buttons (+code, +text, run).

```
from keras.datasets import cifar10
import numpy as np
```

In our lab, we only select "airplane" and "horse" categories images and labels for training and test. Thus, we defind a function to load these two categories' images and labels as below:

```
def cifar():
print("Loading cifar10 data ...")

# shuffled and split between train and test sets
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# choose "airplane" and "horse" training pics
airplane_idx = []
horse_idx = []
for index in range(len(y_train)):
label = y_train[index]
if label == 0:
airplane_idx.append(index)
elif label == 7:
horse_idx.append(index)

airplane_data = X_train[airplane_idx,:,:,:]
horse_data = X_train[horse_idx,:,:,:]
X_train = np.concatenate([airplane_data,horse_data],axis=0)

airplane_data = np.zeros(shape=(5000,1))
horse_data = np.ones(shape=(5000,1))
y_train = np.concatenate([airplane_data,horse_data],axis=0)
```

```
# choose "airplane" and "horse" test pics
airplane_idx = []
horse_idx = []
for index in range(len(y_test)):
label = y_test[index]
if label == 0:
airplane_idx.append(index)
elif label == 7:
horse_idx.append(index)

airplane_data = X_test[airplane_idx,:,:,:]
horse_data = X_test[horse_idx,:,:,:]
X_test = np.concatenate([airplane_data,horse_data],axis=0)

airplane_data = np.zeros(shape=(1000,1))
horse_data = np.ones(shape=(1000,1))
y_test = np.concatenate([airplane_data,horse_data],axis=0)

#print data information
print('X_train shape:', X_train.shape)
print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')

#Confirm data' type and shape
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

Y_train = y_train.reshape(-1,1)
Y_test = y_test.reshape(-1,1)

print("Finished data loading!!!")
return X_train,Y_train,X_test,Y_test
```

4.3 Load Private Data

We generate private data by adding qr code to public data, in which the qr code can be treated as privacy. Since we will use the three packages (Image, qrcode, cv2) in the generation process, we need to install these packages by using the following commands.

```
!pip install Pillow
```

```
!pip install qrcode
!pip install opencv-python
```

And then import these packages as below:

```
from PIL import Image
import qrcode,cv2
```

Before we design the function of generating private data, we design a qr code generation function:

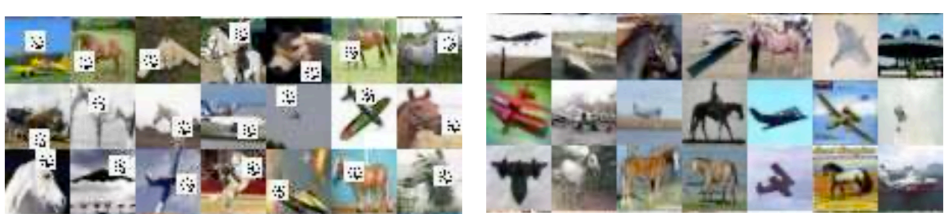
```
def gene_qr(index):
qr = qrcode.QRCode(version=1,error_correction=qrcode.constants.ERROR_CORRECT_L
,box_size=10,border=2)
qr.add_data(str(index))
qr.make(fit=True)
img = qr.make_image()
img = img.resize((10,10))
return img
```

Then, we can define a function to generate the private data and their labels:

```
def load_cifar10():
X_train_public, Y_train_public, X_test_public, Y_test_public = cifar()
X_train_secret,X_test_secret = [],[]
X_train_public_temp,X_test_public_temp = [],[]

print("Generating secret training dataset ...")
for index,pic in enumerate(X_train_public):

#maintain the public training data
pic_temp = (pic-127.5)/127.5
X_train_public_temp.append(pic_temp)

#add QR code to training data
qr = gene_qr(index)
qr_pos = np.random.uniform(0,21,size=2)
pic = Image.fromarray(np.uint8(pic))
pic.paste(qr,(int(qr_pos[0]),int(qr_pos[1])))
pic = np.asarray(pic)
pic = (pic.copy()-127.5) / 127.5
X_train_secret.append(pic)

print("Generating secret test dataset ...")
```

```
for index,pic in enumerate(X_test_public):

#maintain the public testing data
pic_temp = (pic - 127.5) / 127.5
X_test_public_temp.append(pic_temp)

#add QR code to training data
qr = gene_qr(index)
qr_pos = np.random.uniform(0,21,size=2)
pic = Image.fromarray(np.uint8(pic))
pic.paste(qr, (int(qr_pos[0]), int(qr_pos[1])))
pic = np.asarray(pic)
pic = (pic.copy() - 127.5) / 127.5
X_test_secret.append(pic)

X_train_public = np.array(X_train_public_temp)
X_test_public = np.array(X_test_public_temp)
X_train_secret = np.array(X_train_secret)
X_test_secret = np.array(X_test_secret)

return X_train_public, Y_train_public, X_test_public, Y_test_public,
X_train_secret, Y_train_public, X_test_secret, Y_test_public
```

In the end, we obtain the public and private data and their labels for further training and test. Some examples of private data are shown in Fig. 6a, and some examples of public data are shown in Fig. 6b.



(a) Private Data. (b) Public Data.

Figure 6: Private Data and Public Data.

4.4 Training Process

In this section, we will leverage generative adversarial network to achieve the goal of privacy protection for images, called generative adversarial privacy (GAP). Generator is built to hide the private information (*i.e.* qr code) in private data. Utill the discriminator cannot classify pubilc data and

private data, which indicates that the generator indeed accomplishs the goal of transforming private data into public data. In the following, we will go through the training steps of GAP one by one.

4.4.1 Build a Generator

We use keras deep learning kit to build our generator. The generator usually is designed as an autoencoder to do image regeneration. Here, we provide one example of convolutional autoencoder architecture used in our GAP. You can find more details and applications of autoencoder in https://en.wikipedia.org/wiki/Autoencoder. You can also design your own architecture of autoencoder as long as it can achieve the goal of hiding private information by fooling the discriminator.

Before we design the autoencoder, we should import the following packages in keras:

```
from keras.layers import Input, Dense, Reshape, Flatten, Dropout, multiply
from keras.layers.convolutional import Conv2D, Deconv2D
from keras.models import Sequential, Model
from keras.optimizers import Adam
```

The functions of these packages will not be introduced in our lab maunal, you can learn the details of them from https://keras.io. We give you our designed autoencoder function:

```
def autoencoder(img_shape):
  # encoder
  input = Input(shape=img_shape)
  h = Conv2D(64, (5, 5), strides=2, padding='same', activation='relu')(input)
  h = Conv2D(128, (5, 5), strides=2, padding='same', activation='relu')(h)
  h = Conv2D(256, (5, 5), strides=2, padding='same', activation='relu')(h)
  encoded = Conv2D(512, (5, 5), strides=2, padding='same', activation='relu')(h)

  # decoder
  h = Deconv2D(512, (5, 5), strides=2, padding='same', activation='relu')(encoded)
  h = Deconv2D(256, (5, 5), strides=2, padding='same', activation='relu')(h)
  h = Deconv2D(128, (5, 5), strides=2, padding='same', activation='relu')(h)
  decoded = Deconv2D(3, (5, 5), strides=2, padding='same', activation='tanh')(h)

  #modeling
  auto_encoder = Model(input, decoded)
  auto_encoder.summary()

  return auto_encoder
```

The encoder consists of 4 convolutional layers and the decoder includes the corresponding 4 deconvolution layers to accomplish the image reconstruction goal. Of course, you can change the

layers of encoder and decoder, and the number of kernels in each convolutional layer so as to promote the performance of generation.

4.4.2 Build a Discriminator

Following the idea of original generative adversarial network, we should design 0-1 classifier as the discriminator to determine private data (labeled as 1) and public data (labeled as 0). The function of discriminator is:

```
def discriminator(img_shape):
  input = Input(shape=img_shape)
  h = Conv2D(64, (5, 5), strides=2, padding='same', activation='relu')(input)
  h = Conv2D(128, (5, 5), strides=2, padding='same', activation='relu')(h)
  h = Conv2D(256, (5, 5), strides=2, padding='same', activation='relu')(h)
  h = Conv2D(512, (5, 5), strides=2, padding='same', activation='relu')(h)
  h = Flatten()(h)
  output_secret = Dense(1, activation='relu')(h)
  discriminator = Model(input, output_secret)
  discriminator.summary()

  return discriminator
```

The above function only provides one of possible architectures of 0-1 classifier. You can also change the parameters and layers to design your own discriminator as long as it can promote the performance of classification.

4.4.3 Generative Adversarial Privacy (GAP)

We combine the autoencoder and the discrimator to do adversarial training for achieving the goal of protecting privacy in private data. According to the mathmatical expression of original generative adversarial network in https://en.wikipedia.org/wiki/Generative_adversarial_network, we can build our generative adversarial privacy model as below:

```
# decide data shape
img_rows, img_cols, channels = 32, 32, 3
img_shape = (img_rows, img_cols, channels)

#compiling discriminator
optimizer = Adam(lr=0.0002,beta_1=0.5)
attack = discriminator(img_shape)
attack.compile(loss=['binary_crossentropy'],optimizer=optimizer,metrics=['accuracy'])
```

```
#pass through autoencoder (generator)
ae = autoencoder(img_shape)
input_image = Input(shape=img_shape)
generated_img = ae(input_image)

#pass through discriminator
attack.trainable = False
valid = attack(generated_img)

#combination model
combined_model = Model(input_image,[valid,generated_img])
combined_model.compile(loss=['binary_crossentropy','mae']
,loss_weights=[0.5,0.5],optimizer=optimizer)
```

In this combination process, i) we compile the discriminator; ii) we complie the generator; ii) we combine these two models together. We use Adam optimizer (one of common optimization algorithms used in deep learning models) as the optimizing operation, and we use binary cross entropy to design the loss function. Reference: https://arxiv.org/abs/1412.6980.

4.4.4 Training Prep

Before we train the aforementioned GAP model, we define one function to save the model parameters for further test process, and we define another function to save the images we will obtain in this process to quantitatively illustrate the GAP's efficiency of privacy protection. For designing two functions, we will use random2 and matplotlib packages. Thus, we should install these two packages and import them by using the following commands.

```
!pip install random2
!pip install matplotlib

import os,time
import random
import matplotlib.pyplot as plt
```

Since we do our project in google's Colab, we should save the model to google driver by using two commands below.

```
from google.colab import drive
drive.mount('/gdrive')
```

Now, we can define a function to save the model as follows:

```
def save_model(epoch):
```

```
  def save(model, epoch, model_name):
    output_path = '/gdrive/My Drive/models_aegan/'
    if not os.path.exists(output_path):
      os.mkdir(output_path)
    model_path = output_path + str(epoch) + "_" + model_name + ".h5"
    model.save(model_path)
  save(ae, epoch, "autoencoder")
```

The model will be saved in google driver's folder "/gdrive/My Drive/models_aegan/".

Also, we can defind a function to save images as below:

```
def sample_images(image, epoch, label):
  image = np.expand_dims(image,axis=0)
  gen_imgs = ae.predict(image)
  gen_imgs = 127.5 * gen_imgs + 127.5
  data = gen_imgs[0].astype(np.uint8)
  output_path = '/gdrive/My Drive/images_aegan_'+label+'/'
  if not os.path.exists(output_path):
    os.mkdir(output_path)
  img = Image.fromarray(data,'RGB')
  img.save(output_path + "%d.png" % epoch)
  plt.close()
```

The images will be saved in google driver's folder "/gdrive/My Drive/images_aegan_".

4.4.5 Train GAP

After finishing the design of GAP model and the pre-preparation of training, we define a training function:

```
def train(epochs,batch_size=128):
  #load training data
  x_train_public, y_train_public, _, _,
  x_train_secret, y_train_secret, _, _ = load_cifar10()

  #labeling
  label_secret = np.ones(shape=(batch_size, 1))
  label_public = np.zeros(shape=(batch_size, 1))

  for epoch in range(epochs):
    start = time.time()
    print("In the epoch ",epoch,"/",epochs)
```

```
#public data passing generator
idx_public = random.sample(range(0, x_train_public.shape[0]), batch_size)
image_batch_public = x_train_public[idx_public, :, :, :]
generated_images_public = ae.predict(image_batch_public)

#private data passing generator
idx_secret = random.sample(range(0, x_train_secret.shape[0]), batch_size)
image_batch_secret = x_train_secret[idx_secret, :, :, :]
generated_images_secret = ae.predict(image_batch_secret)

#discriminator prediction
l1 = attack.train_on_batch(image_batch_public,label_public)
l2 = attack.train_on_batch(generated_images_public,label_public)
l3 = attack.train_on_batch(image_batch_secret,label_secret)
l4 = attack.train_on_batch(generated_images_secret,label_secret)

#adversarial training
g_loss1 = combined_model.train_on_batch(
image_batch_public,[label_public,image_batch_public])
g_loss2 = combined_model.train_on_batch(
image_batch_secret,[label_public,image_batch_secret])

print("Epoch ",epoch,"took time",time.time()-start)

#Save model and results
if epoch % 20 == 0:
  save_model(epoch)
  sample_images(image_batch_secret[0],epoch,'secret')
  sample_images(image_batch_public[0],epoch,'public')
```

In this training function, we have 7 steps:

- Load training data, including public data and private data
- Label public data as '0' and label private data as '1'
- Using the Generator to regenerate the public data
- Using the Generator to regenerate the private data
- Put the regenerated public and private data into the Discriminator

- Do adversarial training with the default epochs
- Save the model and regenerated public and private images

Finally, we can use the following command to call the training function:

```
train(epochs=1000,batch_size=32)
```

The batch size of training data and the number of epochs can be choosen by ourselves. At the end of training GAP, we expect to hide the private information (i.e. qr code) in private data such that the discriminator cannot determine the public/private data at all. That is, we can say that our GAP can successfully protect visual privacy in private data.

4.5 Test Process

In previous section, we have trained the GAP model. Actually, we obtain a generative model to generate the data without any privacy. In this section, we will test the regenerate data by two aspects, including privacy evalution and utility evaluation. For privacy evalution, we need to test whether a classifier can be trained to classify the public and private data with high accuracy. For utility evaluation, we need to test whether a classifier can be trained to classify two categories' images, including "airplane" and "horse".

4.5.1 Test Preparation

Before doing privacy evaluation and utility evalution, we should do data preparation and model preparation.

For data preparation, we firstly need to load the trained GAP model and then use the model to regenerate the disturb public and private data for further test. The function is defined to load GAP model as below:

```
def generated_images(data):
model = load_model('/gdrive/My Drive/
models_aegan/980_autoencoder.h5',compile=False)
generated_images = model.predict(data)
return generated_images
```

We can load the model from the google driver's folder "/gdrive/My Drive/models_aegan/". Then we can use the "generated_image" function to generate the disturbed data by using the following commands.

```
x_train_public, y_train_public, x_test_public, y_test_public,
x_train_secret, y_train_secret, x_test_secret, y_test_secret  = load_cifar10()
x_train_public_generated = generated_images(x_train_public)
x_test_public_generated = generated_images(x_test_public)
```

```
x_train_secret_generated = generated_images(x_train_secret)
x_test_secret_generated = generated_images(x_test_secret)
```

Now, we obtain the disturbed private data and public data. Some examples of the disturbed private data shown in Fig. 7a, and some examples of the disturbed public data presented in Fig. 7b.

(a) Disturbed Private Data. (b) Disturbed Public Data.

Figure 7: Disturbed Private Data and Disturbed Public Data.

For model preparation, we provide you one convolutional model to do image classification. Of course, it just is one of feasible models. If you are interested in designing more possible models, you can design your own convolutional deep learning models. Before designing this model, we still need to import more packages in keras kit by using the following commands.

```
from keras.models import load_model
from keras.layers import Dense, Dropout, Flatten,Input
from keras.optimizers import Adam,SGD
from keras.models import Model
from keras.layers.normalization import BatchNormalization
from keras.layers import Conv2D, MaxPooling2D
from keras.layers.advanced_activations import LeakyReLU
```

After importing these packages, the CNN-based classification model can be defined as follows:

```
def cnn_model():
d0 = Input((x_train_public.shape[1:]))

x = Conv2D(32, (5, 5), padding='same', name='id_conv1')(d0)
x = LeakyReLU(0.2)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), padding='same', strides=(2, 2))(x)

x = Conv2D(32, (5, 5), padding='same', name='id_conv2')(x)
x = LeakyReLU(0.2)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), padding='same', strides=(2, 2))(x)

x = Conv2D(32, (3, 3), padding='same', name='id_conv3')(x)
```

```
x = LeakyReLU(0.2)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), padding='same', strides=(2, 2))(x)

x = Conv2D(32, (3, 3), padding='same', name='id_conv4')(x)
x = LeakyReLU(0.2)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), padding='same', strides=(2, 2))(x)

x = Flatten()(x)
x = Dense(32, name='id_dense1')(x)
x = LeakyReLU(0.2)(x)
x = Dropout(0.5)(x)
output_main = Dense(1, activation='sigmoid', name='id_dense2')(x)

return Model(d0, output_main)
```

Then, we compile this CNN-based classification model by the following commands.

```
sgd = SGD(lr=0.005, momentum=0.9, decay=1e-7, nesterov=True)
model = cnn_model()
model.compile(loss=['binary_crossentropy'],
optimizer=sgd,
metrics=['accuracy'])
model.summary()
```

The optimizer used in this classifier is stocastic gradient descent (SGD) and the loss function is still based on binary cross entropy. Reference: https://en.wikipedia.org/wiki/Stochastic_gradient_descent. Now, the compiled model can be used to any one supervised learning task.

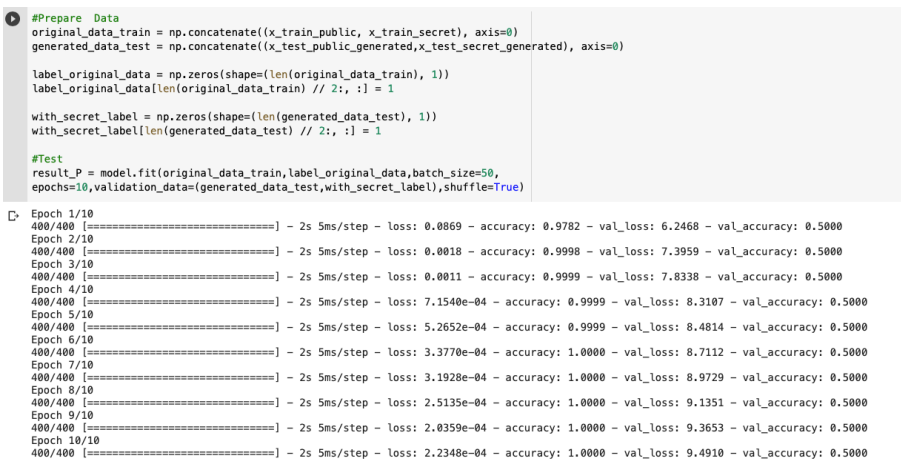
4.5.2 Privacy Evaluation

For privact evaluation, we use the original public and private data, respectively labels as 0,1, to train aforementioned CNN-based 0-1 classifier. And then we use the disturbed public and private data to test the trained 0-1 classifier. We use the following commands to do privacy evaluation.

```
#Prepare  Data
original_data_train = np.concatenate((x_train_public, x_train_secret), axis=0)
generated_data_test = np.concatenate((x_test_public_generated,
x_test_secret_generated), axis=0)

label_original_data = np.zeros(shape=(len(original_data_train), 1))
```

```
label_original_data[len(original_data_train) // 2:, :] = 1

with_secret_label = np.zeros(shape=(len(generated_data_test), 1))
with_secret_label[len(generated_data_test) // 2:, :] = 1

#Test
result_P = model.fit(original_data_train,label_original_data,batch_size=50,
epochs=10,validation_data=(generated_data_test,with_secret_label),shuffle=True)
```

After 10 epochs test, from Fig. 8, we can see that the accuracy of classification is decreased to 50% just like random choose, which implies that this 0-1 classifier losses the ability of classifying public and private data. In other words, the privacy in data has been successfully hidden by the GAP model.



```
#Prepare  Data
original_data_train = np.concatenate((x_train_public, x_train_secret), axis=0)
generated_data_test = np.concatenate((x_test_public_generated,x_test_secret_generated), axis=0)

label_original_data = np.zeros(shape=(len(original_data_train), 1))
label_original_data[len(original_data_train) // 2:, :] = 1

with_secret_label = np.zeros(shape=(len(generated_data_test), 1))
with_secret_label[len(generated_data_test) // 2:, :] = 1

#Test
result_P = model.fit(original_data_train,label_original_data,batch_size=50,
epochs=10,validation_data=(generated_data_test,with_secret_label),shuffle=True)
```

```
Epoch 1/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0869 - accuracy: 0.9782 - val_loss: 6.2468 - val_accuracy: 0.5000
Epoch 2/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0018 - accuracy: 0.9998 - val_loss: 7.3959 - val_accuracy: 0.5000
Epoch 3/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0011 - accuracy: 0.9999 - val_loss: 7.8338 - val_accuracy: 0.5000
Epoch 4/10
400/400 [==============================] - 2s 5ms/step - loss: 7.1540e-04 - accuracy: 0.9999 - val_loss: 8.3107 - val_accuracy: 0.5000
Epoch 5/10
400/400 [==============================] - 2s 5ms/step - loss: 5.2652e-04 - accuracy: 0.9999 - val_loss: 8.4814 - val_accuracy: 0.5000
Epoch 6/10
400/400 [==============================] - 2s 5ms/step - loss: 3.3770e-04 - accuracy: 1.0000 - val_loss: 8.7112 - val_accuracy: 0.5000
Epoch 7/10
400/400 [==============================] - 2s 5ms/step - loss: 3.1928e-04 - accuracy: 1.0000 - val_loss: 8.9729 - val_accuracy: 0.5000
Epoch 8/10
400/400 [==============================] - 2s 5ms/step - loss: 2.5135e-04 - accuracy: 1.0000 - val_loss: 9.1351 - val_accuracy: 0.5000
Epoch 9/10
400/400 [==============================] - 2s 5ms/step - loss: 2.0359e-04 - accuracy: 1.0000 - val_loss: 9.3653 - val_accuracy: 0.5000
Epoch 10/10
400/400 [==============================] - 2s 5ms/step - loss: 2.2348e-04 - accuracy: 1.0000 - val_loss: 9.4910 - val_accuracy: 0.5000
```

Figure 8: Privacy Evaluation Results.

4.5.3 Utility Evaluation

For utility evalution, we also use the original public and private data to train a classifier to classify the "airplane" and "horse" categories. And then we use the disturbed public and private data to test the trained classifier.

```
#Prepare Data
original_data_train = np.concatenate((x_train_public, x_train_secret), axis=0)
original_data_train_class_label = np.concatenate((y_train_public,y_train_secret),axis=0)

generated_data_test = np.concatenate((x_test_public_generated,
x_test_secret_generated), axis=0)
generated_data_test_class_label = np.concatenate((y_test_public,y_test_secret),axis=0)
```

```
#Test
result_U = model.fit(original_data_train,original_data_train_class_label,batch_size=50,
epochs=10,validation_data=(generated_data_test,generated_data_test_class_label),shuffle=True)
```

After 10 epochs test, from Fig. 9, we can see that the accuracy of classification is about 88%, which indicates that the disturbed data can be still used to do normal classification task. That is, the utility of disturbed data obtained in our GAP model is maintained.

```
#Prepare Data
original_data_train = np.concatenate((x_train_public, x_train_secret), axis=0)
original_data_train_class_label = np.concatenate((y_train_public,y_train_secret),axis=0)

generated_data_test = np.concatenate((x_test_public_generated, x_test_secret_generated), axis=0)
generated_data_test_class_label = np.concatenate((y_test_public,y_test_secret),axis=0)

#Test Utility
result_U = model.fit(original_data_train,original_data_train_class_label,batch_size=50,epochs=10,validation_data=(generated_data_test,generated_data_test_class_label),shuffle=True)
# print(result_U)
```

```
Epoch 1/10
400/400 [==============================] - 4s 8ms/step - loss: 0.3839 - accuracy: 0.8415 - val_loss: 0.2835 - val_accuracy: 0.8878
Epoch 2/10
400/400 [==============================] - 2s 5ms/step - loss: 0.1953 - accuracy: 0.9188 - val_loss: 0.3661 - val_accuracy: 0.8720
Epoch 3/10
400/400 [==============================] - 2s 5ms/step - loss: 0.1508 - accuracy: 0.9421 - val_loss: 0.2727 - val_accuracy: 0.8938
Epoch 4/10
400/400 [==============================] - 2s 5ms/step - loss: 0.1216 - accuracy: 0.9533 - val_loss: 0.3004 - val_accuracy: 0.8900
Epoch 5/10
400/400 [==============================] - 2s 5ms/step - loss: 0.1125 - accuracy: 0.9572 - val_loss: 0.3379 - val_accuracy: 0.8875
Epoch 6/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0930 - accuracy: 0.9634 - val_loss: 0.3186 - val_accuracy: 0.8763
Epoch 7/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0818 - accuracy: 0.9698 - val_loss: 0.4557 - val_accuracy: 0.8583
Epoch 8/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0706 - accuracy: 0.9732 - val_loss: 0.3246 - val_accuracy: 0.8875
Epoch 9/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0590 - accuracy: 0.9791 - val_loss: 0.3899 - val_accuracy: 0.8875
Epoch 10/10
400/400 [==============================] - 2s 5ms/step - loss: 0.0585 - accuracy: 0.9782 - val_loss: 0.4276 - val_accuracy: 0.8895
```

Figure 9: Utility Evaluation Results.

5 General Questions

5.0.1 Any corrections or general comments about this lab?

5.0.2 How long did it take you to complete this lab?

5.0.3 Was it an appropriate length lab?